

CSS Media Queries Tutorial: Complete Guide

CSS Learning Resources

May 9, 2026

Contents

1	Introduction	2
1.1	Why Media Queries Matter	2
2	Media Query Fundamentals	3
2.1	Basic Syntax	3
2.2	Media Query Structure	3
3	Media Types	4
3.1	Available Media Types	4
3.2	Media Type Examples	4
4	Common Media Features	5
4.1	Width-Based Features	5
4.2	Height-Based Features	5
4.3	Orientation	6
4.4	Device Pixel Ratio	6
5	Common Breakpoints	7
5.1	Standard Device Breakpoints	7
5.2	Mobile-First Approach	7
5.3	Desktop-First Approach	8
6	Logical Operators	9
6.1	AND Operator	9
6.2	OR Operator (Comma)	9
6.3	NOT Operator	9
7	Advanced Media Features	11
7.1	Hover Capability	11
7.2	Pointer Type	11
7.3	Display Quality	11
8	Real-World Examples	13
8.1	Example 1: Responsive Container	13
8.2	Example 2: Responsive Navigation	13
8.3	Example 3: Responsive Grid Layout	14
8.4	Example 4: Touch-Optimized Interface	15
8.5	Example 5: Print Styles	15

8.6	Example 6: Retina Display Optimization	16
9	Common Media Query Patterns	17
9.1	Mobile-First Pattern	17
9.2	Tablet-Specific Styles	17
9.3	Orientation-Based Layout	17
10	Browser Support and Best Practices	18
10.1	Browser Compatibility	18
10.2	Best Practices	18
11	Common Mistakes	19
11.1	Mistake 1: Wrong Breakpoint Order	19
11.2	Mistake 2: Missing Viewport Meta Tag	19
11.3	Mistake 3: Overlapping Breakpoints	20
11.4	Mistake 4: Fixed Widths in Responsive Layout	20
12	Quick Reference	21
12.1	Common Media Queries	21
12.2	Production Checklist	21
13	Summary	23

Chapter 1

Introduction

Media queries are a cornerstone of responsive web design, enabling developers to create layouts that adapt seamlessly across devices of all sizes. They allow CSS rules to be applied conditionally based on device characteristics such as screen width, height, orientation, and other media features. Understanding media queries is essential for building modern, user-friendly web applications that work perfectly on smartphones, tablets, laptops, and large displays.

1.1 Why Media Queries Matter

- **Responsive Design:** Create layouts that adapt to any screen size
- **Mobile-First Development:** Design for small screens first, enhance for larger
- **Device Optimization:** Tailor experiences for phones, tablets, desktops, and watches
- **Performance:** Reduce unnecessary content on smaller devices
- **User Experience:** Ensure readability and usability across all devices
- **Future-Proof:** Handle new device sizes without code changes
- **Accessibility:** Adjust layouts for different interaction modes

Chapter 2

Media Query Fundamentals

2.1 Basic Syntax

Media queries follow a simple structure:

```
1 @media (media-feature) {  
2   /* CSS rules applied when condition is true */  
3   selector {  
4     property: value;  
5   }  
6 }
```

2.2 Media Query Structure

A complete media query consists of:

```
1 /* @media + media type + media feature + CSS rules */  
2 @media screen and (max-width: 768px) {  
3   body {  
4     font-size: 14px;  
5   }  
6 }
```

- **@media:** Media query keyword
- **Media Type:** all, screen, print, speech (optional)
- **Media Features:** Conditions in parentheses
- **CSS Rules:** Styles applied when condition is true

Chapter 3

Media Types

3.1 Available Media Types

Type	Description
all	Applies to all devices (default)
screen	Computer screens, tablets, smart-phones
print	Printers and print preview
speech	Screen readers and voice assistants

3.2 Media Type Examples

```
1  /* Applies to all devices */
2  @media all {
3    body { color: black; }
4  }
5
6  /* Applies to screens only */
7  @media screen {
8    body { font-size: 16px; }
9  }
10
11 /* Applies to printed documents */
12 @media print {
13   body { color: black; background: white; }
14   .no-print { display: none; }
15 }
16
17 /* Applies to screen readers */
18 @media speech {
19   body { volume: medium; }
20 }
```

Chapter 4

Common Media Features

4.1 Width-Based Features

Width-based media features are the most commonly used for responsive design.

```
1  /* Maximum width (mobile-first) */
2  @media (max-width: 768px) {
3      .container { width: 100%; }
4  }
5
6  /* Minimum width (desktop-first) */
7  @media (min-width: 1024px) {
8      .container { width: 960px; }
9  }
10
11 /* Exact width match */
12 @media (width: 768px) {
13     /* CSS rules */
14 }
15
16 /* Width range */
17 @media (min-width: 768px) and (max-width: 1024px) {
18     .container { width: 750px; }
19 }
```

4.2 Height-Based Features

```
1  /* Maximum viewport height */
2  @media (max-height: 600px) {
3      header { height: 40px; }
4  }
5
6  /* Minimum viewport height */
7  @media (min-height: 700px) {
8      .sidebar { position: fixed; }
9  }
10
11 /* Aspect ratio based on height */
```

```
12 @media (max-height: 700px) {  
13     nav { display: none; }  
14 }
```

4.3 Orientation

```
1 /* Portrait orientation (height > width) */  
2 @media (orientation: portrait) {  
3     .layout { flex-direction: column; }  
4 }  
5  
6 /* Landscape orientation (width > height) */  
7 @media (orientation: landscape) {  
8     .layout { flex-direction: row; }  
9 }
```

4.4 Device Pixel Ratio

```
1 /* Retina displays (2x pixel ratio) */  
2 @media (min-device-pixel-ratio: 2) {  
3     .icon {  
4         background-image: url('icon@2x.png');  
5     }  
6 }  
7  
8 /* High DPI screens */  
9 @media (min-device-pixel-ratio: 1.5) {  
10     .image {  
11         background-size: contain;  
12     }  
13 }
```


Chapter 5

Common Breakpoints

5.1 Standard Device Breakpoints

Device	Width Range
Mobile Small	320px – 480px
Mobile	480px – 768px
Tablet	768px – 1024px
Desktop	1024px – 1440px
Large Desktop	1440px+

5.2 Mobile-First Approach

```
1  /* Base styles for mobile (smallest screens) */
2  body { font-size: 14px; }
3  .container { width: 100%; }
4
5  /* Tablet breakpoint */
6  @media (min-width: 768px) {
7      body { font-size: 16px; }
8      .container { width: 750px; }
9  }
10
11 /* Desktop breakpoint */
12 @media (min-width: 1024px) {
13     body { font-size: 18px; }
14     .container { width: 960px; }
15 }
16
17 /* Large desktop breakpoint */
18 @media (min-width: 1440px) {
19     body { font-size: 20px; }
20     .container { width: 1320px; }
21 }
```

5.3 Desktop-First Approach

```
1  /* Base styles for large desktop */
2  body { font-size: 18px; }
3  .container { width: 1200px; }
4
5  /* Large tablet breakpoint */
6  @media (max-width: 1024px) {
7      body { font-size: 16px; }
8      .container { width: 960px; }
9  }
10
11 /* Small tablet breakpoint */
12 @media (max-width: 768px) {
13     body { font-size: 15px; }
14     .container { width: 100%; }
15 }
16
17 /* Mobile breakpoint */
18 @media (max-width: 480px) {
19     body { font-size: 14px; }
20 }
```

Chapter 6

Logical Operators

6.1 AND Operator

Combine multiple conditions using `and`:

```
1 /* Apply when both conditions are true */
2 @media (min-width: 768px) and (max-width: 1024px) {
3   .container { width: 750px; }
4 }
5
6 /* Multiple conditions */
7 @media screen and (min-width: 768px)
8       and (orientation: landscape) {
9   .layout { display: grid; }
10 }
```

6.2 OR Operator (Comma)

Apply styles to multiple media queries:

```
1 /* Apply to portrait OR mobile screens */
2 @media (orientation: portrait),
3       (max-width: 600px) {
4   .sidebar { display: none; }
5 }
```

6.3 NOT Operator

Exclude specific conditions:

```
1 /* Apply to everything except print */
2 @media not print {
3   .screen-only { display: block; }
4 }
5
6 /* Modern syntax with parentheses */
7 @media not (orientation: portrait) {
8   .landscape { display: flex; }
```

9 }

Chapter 7

Advanced Media Features

7.1 Hover Capability

```
1 /* Devices that support hover (desktop) */
2 @media (hover: hover) {
3     button:hover { background-color: #0056b3; }
4 }
5
6 /* Touch devices without hover */
7 @media (hover: none) {
8     button { padding: 12px; }
9 }
```

7.2 Pointer Type

```
1 /* Precise pointer (mouse) */
2 @media (pointer: fine) {
3     .button { padding: 8px 12px; }
4 }
5
6 /* Coarse pointer (touch) */
7 @media (pointer: coarse) {
8     .button { padding: 12px 16px; }
9 }
10
11 /* No pointer available */
12 @media (pointer: none) {
13     .button { display: none; }
14 }
```

7.3 Display Quality

```
1 /* Monochrome displays */
2 @media (monochrome) {
3     body { color: #000; }
```

```
4  }
5
6  /* Color displays */
7  @media (color) {
8      body { color: #333; }
9  }
10
11 /* Minimum color bits per pixel */
12 @media (min-color: 8) {
13     .gradient { background: linear-gradient(...); }
14 }
```

Chapter 8

Real-World Examples

8.1 Example 1: Responsive Container

```
1 .container {
2     width: 100%;
3     padding: 10px;
4     margin: 0 auto;
5 }
6
7 @media (min-width: 640px) {
8     .container {
9         width: 640px;
10        padding: 15px;
11    }
12 }
13
14 @media (min-width: 768px) {
15     .container {
16         width: 750px;
17    }
18 }
19
20 @media (min-width: 1024px) {
21     .container {
22         width: 960px;
23    }
24 }
25
26 @media (min-width: 1280px) {
27     .container {
28         width: 1200px;
29    }
30 }
```

8.2 Example 2: Responsive Navigation

```
1 /* Mobile navigation */
```

```
2 nav {
3   display: none;
4   position: fixed;
5   width: 100%;
6   background: white;
7   z-index: 1000;
8 }
9
10 nav.active {
11   display: block;
12 }
13
14 nav ul {
15   flex-direction: column;
16   padding: 0;
17   margin: 0;
18 }
19
20 nav li {
21   border-bottom: 1px solid #ddd;
22 }
23
24 /* Tablet and up */
25 @media (min-width: 768px) {
26   nav {
27     display: flex;
28     position: static;
29   }
30
31   nav ul {
32     flex-direction: row;
33     gap: 2rem;
34   }
35
36   nav li {
37     border-bottom: none;
38   }
39 }
```

8.3 Example 3: Responsive Grid Layout

```
1 .grid {
2   display: grid;
3   grid-template-columns: 1fr;
4   gap: 1rem;
5 }
6
7 /* Tablet: 2 columns */
8 @media (min-width: 768px) {
9   .grid {
10     grid-template-columns: repeat(2, 1fr);
11   }
12 }
```



```
12 }
13
14 /* Desktop: 3 columns */
15 @media (min-width: 1024px) {
16   .grid {
17     grid-template-columns: repeat(3, 1fr);
18     gap: 2rem;
19   }
20 }
21
22 /* Large: 4 columns */
23 @media (min-width: 1440px) {
24   .grid {
25     grid-template-columns: repeat(4, 1fr);
26   }
27 }
```

8.4 Example 4: Touch-Optimized Interface

```
1 /* Desktop with hover support */
2 @media (hover: hover) {
3   .button {
4     padding: 8px 16px;
5     background: #007bff;
6   }
7
8   .button:hover {
9     background: #0056b3;
10  }
11 }
12
13 /* Touch devices */
14 @media (hover: none) and (pointer: coarse) {
15   .button {
16     padding: 12px 20px;
17     min-height: 44px;
18     font-size: 16px;
19   }
20
21   .button:active {
22     background: #0056b3;
23   }
24 }
```

8.5 Example 5: Print Styles

```
1 /* Screen styles */
2 @media screen {
3   .navbar { position: fixed; }
4   .sidebar { display: block; }
```

```
5  .print-hide { display: none; }
6  }
7
8  /* Print styles */
9  @media print {
10   body { font-size: 12pt; }
11   .navbar { display: none; }
12   .sidebar { display: none; }
13   .main { width: 100%; }
14   a { text-decoration: underline; }
15
16   .page-break {
17     page-break-after: always;
18   }
19 }
```

8.6 Example 6: Retina Display Optimization

```
1  /* Standard displays */
2  .logo {
3    background-image: url('logo.png');
4    background-size: contain;
5    width: 200px;
6    height: 100px;
7  }
8
9  /* Retina displays (2x pixel ratio) */
10 @media (min-device-pixel-ratio: 2),
11        (-webkit-min-device-pixel-ratio: 2) {
12   .logo {
13     background-image: url('logo@2x.png');
14   }
15 }
16
17 /* High-end displays (3x pixel ratio) */
18 @media (min-device-pixel-ratio: 3),
19        (-webkit-min-device-pixel-ratio: 3) {
20   .logo {
21     background-image: url('logo@3x.png');
22   }
23 }
```

Chapter 9

Common Media Query Patterns

9.1 Mobile-First Pattern

```
1  /* Start with mobile defaults */
2  .sidebar { display: none; }
3  .main { width: 100%; }
4
5  /* Show sidebar on tablets and up */
6  @media (min-width: 1024px) {
7    .sidebar { display: block; width: 250px; }
8    .main { width: calc(100% - 250px); }
9  }
```

9.2 Tablet-Specific Styles

```
1  /* Only apply between tablet sizes */
2  @media (min-width: 768px) and (max-width: 1023px) {
3    .container { width: 100%; padding: 1rem; }
4  }
```

9.3 Orientation-Based Layout

```
1  /* Portrait mode */
2  @media (orientation: portrait) {
3    .layout { flex-direction: column; }
4  }
5
6  /* Landscape mode */
7  @media (orientation: landscape) {
8    .layout { flex-direction: row; }
9    .sidebar { width: 200px; }
10 }
```

Chapter 10

Browser Support and Best Practices

10.1 Browser Compatibility

Feature	Support
Basic Media Queries	Chrome 4+, Firefox 3.5+, Safari 3.2+, IE 9+
Media Features	Modern browsers (95%+)
Device Pixel Ratio	Chrome 29+, Firefox 16+, Safari 5.1+
Hover Media Query	Chrome 38+, Firefox 64+, Safari 9+
Touch Media Query	Chrome 41+, Firefox 64+, Safari 13+

10.2 Best Practices

1. **Mobile-First:** Design for mobile first, then enhance for larger screens
2. **Test on Real Devices:** Always test on actual phones, tablets, and desktops
3. **Use Standard Breakpoints:** Follow common breakpoints for consistency
4. **Avoid Device-Specific:** Don't hardcode specific devices
5. **Touch-Friendly:** Use larger touch targets (44px minimum)
6. **Performance:** Optimize images and content for smaller screens
7. **Progressive Enhancement:** Start with basic styles, enhance with queries
8. **Keyboard Navigation:** Ensure media queries don't break keyboard access
9. **Test Print Styles:** Always test print media queries
10. **Use CSS Frameworks:** Consider frameworks like Bootstrap, Tailwind

Chapter 11

Common Mistakes

11.1 Mistake 1: Wrong Breakpoint Order

Wrong:

```
1 @media (min-width: 1024px) {  
2   .sidebar { width: 250px; }  
3 }  
4  
5 @media (min-width: 768px) {  
6   .sidebar { width: 150px; }  
7 }
```

Correct:

```
1 @media (min-width: 768px) {  
2   .sidebar { width: 150px; }  
3 }  
4  
5 @media (min-width: 1024px) {  
6   .sidebar { width: 250px; }  
7 }
```

11.2 Mistake 2: Missing Viewport Meta Tag

Wrong:

```
1 <!-- Missing viewport meta tag -->  
2 <head>  
3   <link rel="stylesheet" href="style.css">  
4 </head>
```

Correct:

```
1 <head>  
2   <meta name="viewport" content="width=device-width, initial-scale=1">  
3   <link rel="stylesheet" href="style.css">  
4 </head>
```

11.3 Mistake 3: Overlapping Breakpoints

Wrong:

```
1 @media (max-width: 768px) { /* 0-768px */ }
2 @media (max-width: 1024px) { /* 0-1024px overlaps above */ }
```

Correct:

```
1 @media (max-width: 767px) { /* 0-767px */ }
2 @media (min-width: 768px) and (max-width: 1023px) { /* 768-1023px */ }
3 @media (min-width: 1024px) { /* 1024px+ */ }
```

11.4 Mistake 4: Fixed Widths in Responsive Layout

Wrong:

```
1 .container {
2   width: 1024px; /* Fixed, doesn't respond */
3 }
```

Correct:

```
1 .container {
2   width: 100%;
3   max-width: 1024px;
4   padding: 1rem;
5 }
```

Chapter 12

Quick Reference

12.1 Common Media Queries

```
1  /* Mobile only */
2  @media (max-width: 767px) { }
3
4  /* Tablet and up */
5  @media (min-width: 768px) { }
6
7  /* Desktop and up */
8  @media (min-width: 1024px) { }
9
10 /* Large desktop */
11 @media (min-width: 1440px) { }
12
13 /* Landscape orientation */
14 @media (orientation: landscape) { }
15
16 /* Touch devices */
17 @media (hover: none) and (pointer: coarse) { }
18
19 /* Print */
20 @media print { }
21
22 /* Range query */
23 @media (768px <= width <= 1024px) { }
```

12.2 Production Checklist

- Add viewport meta tag to HTML
- Use mobile-first approach
- Test on actual devices
- Verify all breakpoints work
- Optimize images for each size

- Ensure touch targets are 44px minimum
- Test print styles
- Verify keyboard navigation
- Check performance on mobile
- Test orientation changes

Chapter 13

Summary

Media queries are essential for creating responsive, adaptive web experiences. Key takeaways:

1. Media queries apply CSS conditionally based on device features
2. Use common breakpoints: 768px (tablet), 1024px (desktop)
3. Start with mobile-first design, then enhance for larger screens
4. Always include viewport meta tag in HTML
5. Test on real devices, not just browser DevTools
6. Use min-width for progressive enhancement
7. Combine multiple conditions with **and**
8. Handle touch devices with appropriate target sizes
9. Optimize images for different pixel ratios
10. Consider performance impact on mobile devices

Master CSS media queries to create interfaces that look great and work flawlessly on every device, from smartphones to large desktop displays.